

Criterion B: Design

Preliminary designs and familiarizing with front end:

Picking up understanding of HTML and CSS elements

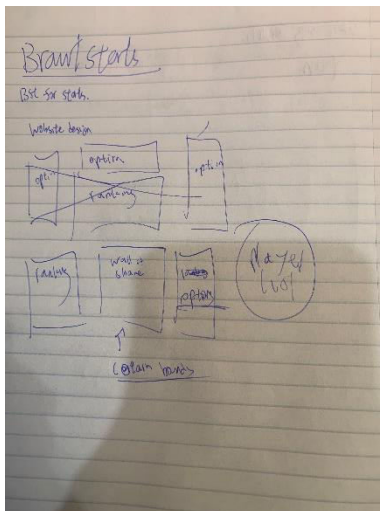
Heading test

test

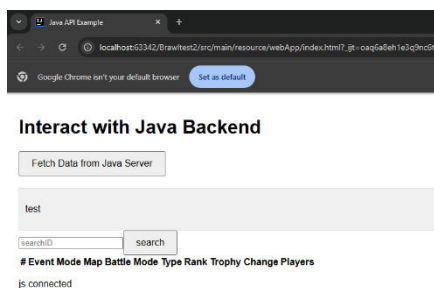


let's go gambling

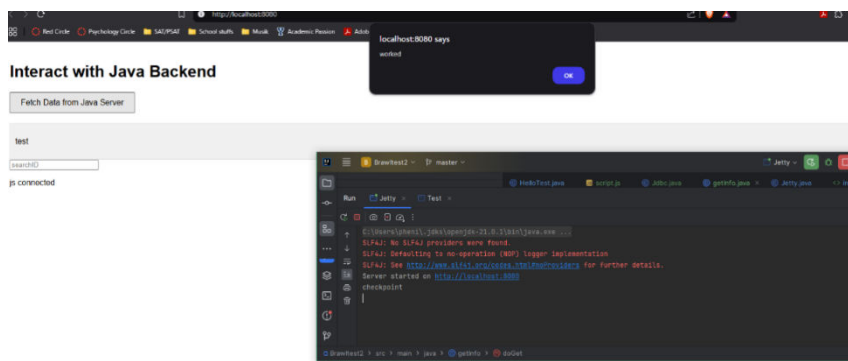
Once I finished learning the basics of, I drafted an initial UI



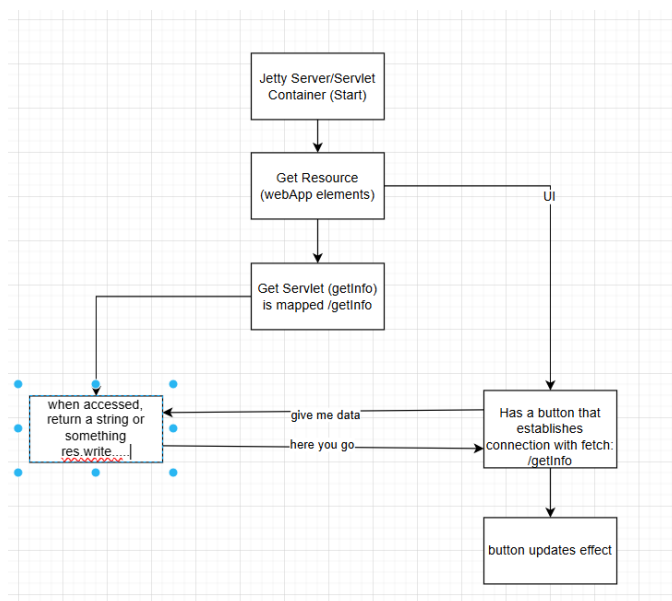
I successfully connected the HTML with JavaScript (but not Java)



After a bit, HTML also linked with Java



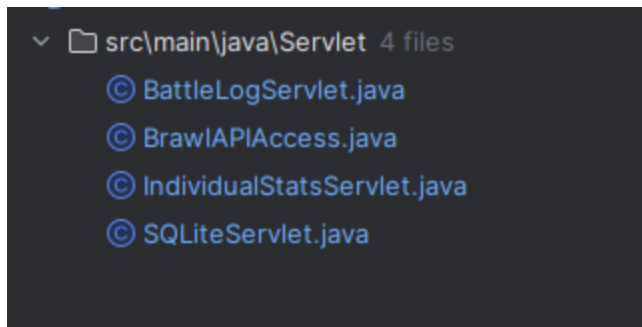
This initial flowchart generalizes what I do to connect the frontend with the back end



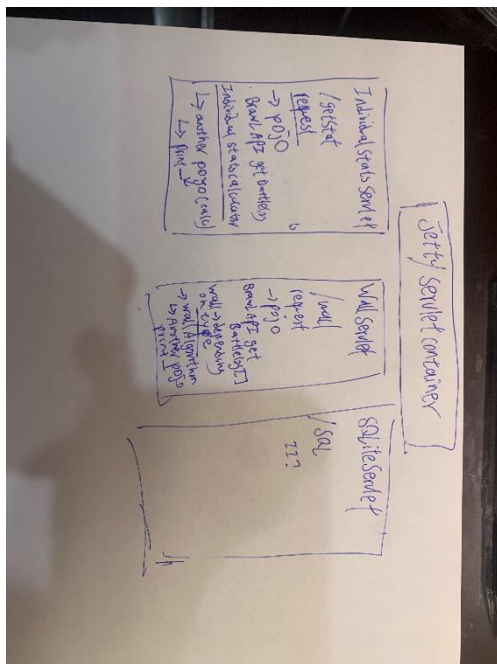
Progressive designs:

Jetty:

Server and Servlet container, it is what allows the backend to function, the final product should contain at least 3 servlets: one for Wall, one for IndividualStats, one for Settings. I have tried over 4 different Server/Servlet containers including Apache tomcat, but Jetty is the most intuitive and simple one to implement and understand. Because my server structure is rather simple, I do not see the need for a more complex structure like SpringBoot.



All these servlets uses doGet requests instead of doPost because doPost in my structure is unnecessary and would require more POJOS which make the program look more complicated.



This is an example of my initial take on WallServlet processing request:

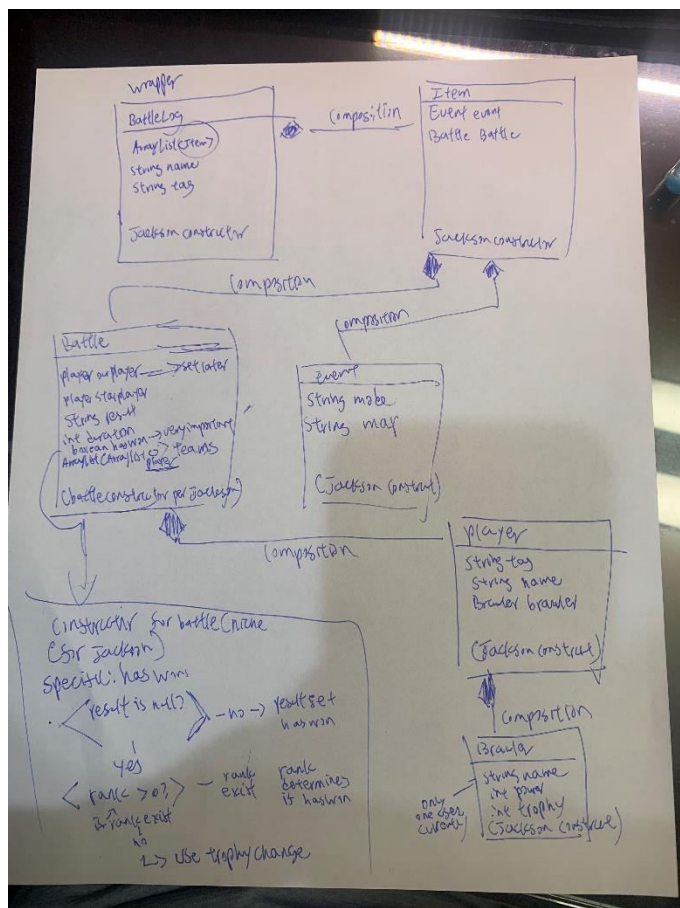


POJO:

POJO in java helps encapsulate and deserialize JSON files into plain-old java object. In my backend are 2 POJO types, one for receiving from Brawl stars API and sending to the calculation algorithms, another for sending to JS after backend is done with processing the calculations.

POJO type 1 for BattleLog:

For the full JSON file please go to <https://developer.brawlstars.com/#/login> (it's 70 pages long per player)

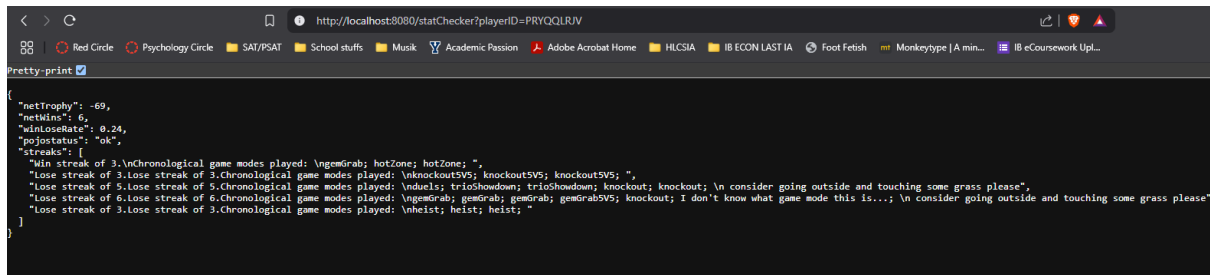


The POJO is instantiated/JSON deserialized by the Jackson packet. Same with serialization.

POJO type 2:

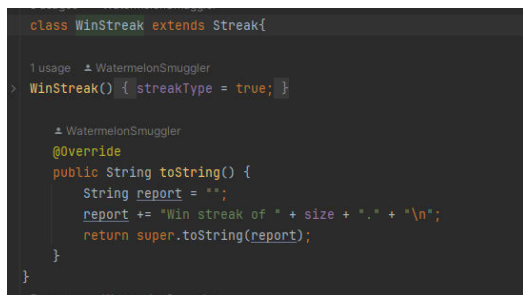
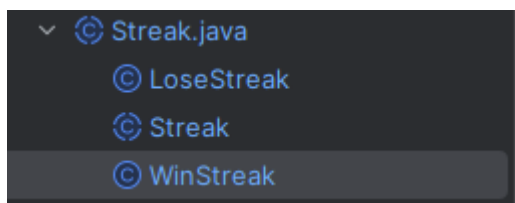
Because it is the outcome of my algorithm, it only need to be one class, and the structure much less complex:

Sample JSON result during testing:



```
{
  "netTrophy": -69,
  "netWins": 6,
  "winLossRate": 0.24,
  "pojostatus": "ok",
  "streaks": [
    "Win streak of 3.Chronological game modes played: \nGemGrab; hotZone; hotZone;",
    "Lose streak of 3.Chronological game modes played: \nknockoutSV5; knockoutSV5; ",
    "Lose streak of 5.Lose streak of 5.Chronological game modes played: \nduels; trioShowdown; trioShowdown; knockout; knockout; \n consider going outside and touching some grass please",
    "Lose streak of 6.Lose streak of 6.Chronological game modes played: \nGemGrab; gemGrab; gemGrab; gemGrabSV5; knockout; I don't know what game mode this is...; \n consider going outside and touching some grass please",
    "Lose streak of 3.Lose streak of 3.Chronological game modes played: \nheist; heist; heist;"
  ]
}
```

For the sake of POJOs, the Streak classes for recording streaks in my program was also separated into WinStreak and LoseStreak implementing the Streak class, this is to ensure the Wall of honor and shame can properly identify between a win and lose streak, and POJO has to know as well. I recognize them as 2 distinct classes with different features.



```
class WinStreak extends Streak{
  1 usage 1 WatermelonSmuggler
  > WinStreak() { streakType = true; }

  1 WatermelonSmuggler
  @Override
  public String toString() {
    String report = "";
    report += "Win streak of " + size + "." + "\n";
    return super.toString(report);
  }
}
```

POJO continued + Connectivity tests

JSONs like above contain processed information JavaScript uses to convert into tables in its fetching lambda functions:

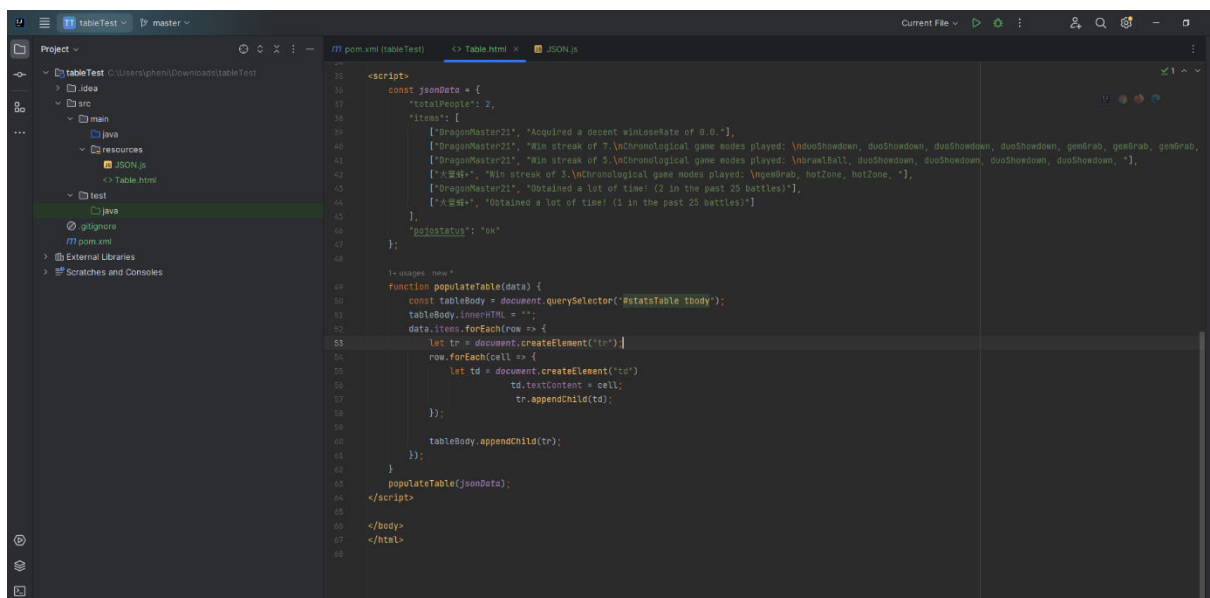
```

}) Promise<any>
.then(data =>{
    document.getElementById( elementId: "numberOfPlayer").innerHTML = data.totalPeople;
    //because it's an ID, statsTable has to be preceded by a #
    const tableBody : Element = document.querySelector( selectors: "#statsTable tbody");
    tableBody.innerHTML = "";
    data.items.forEach(row => {
        let tr : HTMLTableRowElement = document.createElement( tagName: "tr");

        row.forEach(cell => {
            let td : HTMLTableCellElement = document.createElement( tagName: "td");
            td.textContent = cell;
            tr.appendChild(td);
        });
    });
});

```

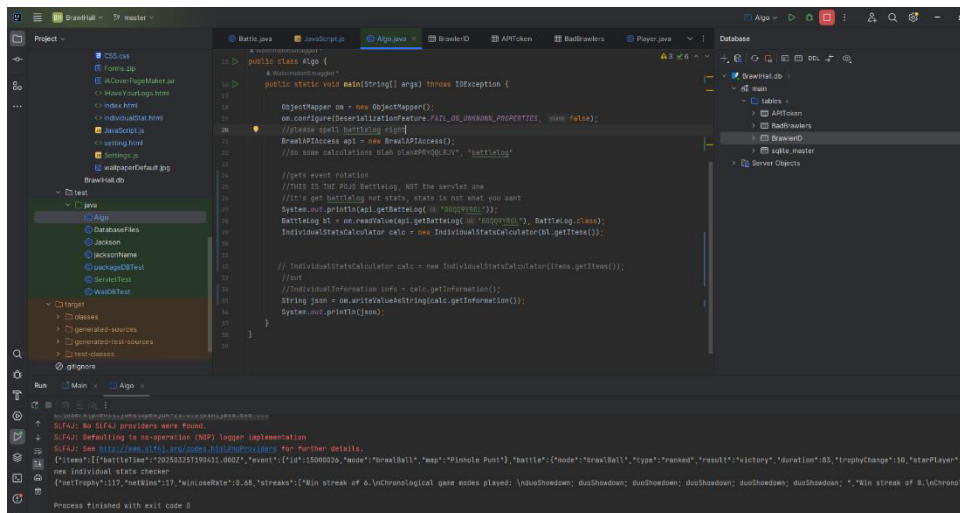
Because it is a server, testing is impossibly difficult when servlet is open, therefore I used prebuilt JSON to test the JavaScript table building functions:



Which worked quite well:

Player	Achievement
DragonMaster21	Acquired a decent winLoseRate of 0.0.
DragonMaster21	Win streak of 7. Chronological game modes played: duoShowdown, duoShowdown, duoShowdown, duoShowdown, gemGrab, gemGrab, gemGrab,
DragonMaster21	Win streak of 5. Chronological game modes played: brawlBall, duoShowdown, duoShowdown, duoShowdown, duoShowdown,
大皇蜂+	Win streak of 3. Chronological game modes played: gemGrab, hotZone, hotZone,
DragonMaster21	Obtained a lot of time! (2 in the past 25 battles)
大皇蜂+	Obtained a lot of time! (1 in the past 25 battles)

For backend tests, I used Maven's built in test environments, which had faithfully debugged the process between retrieving the JSON and POJO from Brawl Stars API to the type 2 POJO of processed information ready to be displayed/converted to a table on the HTML:

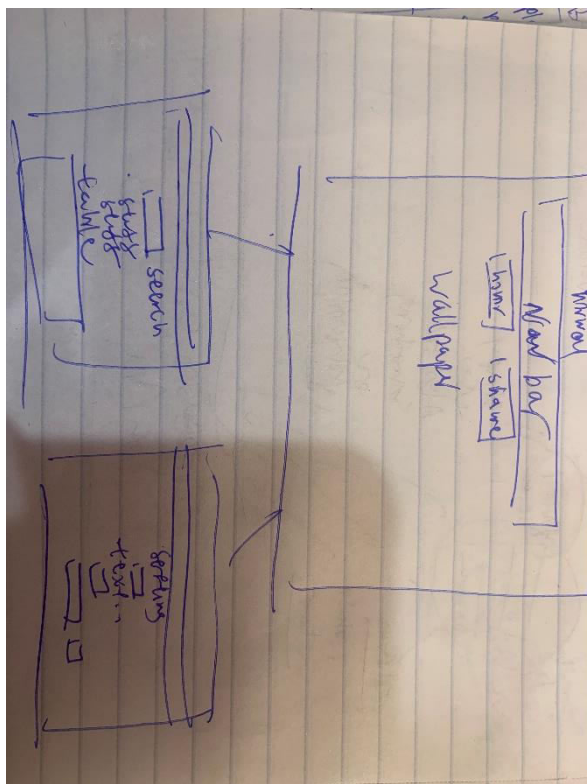


For the record, I restarted my project twice midway and ended up using Maven, it made adding dependencies (Jetty, Xerial for DB, SQL, Jackson) much easier. Thanks to Maven, it saved me time downloading the right version that is compatible with my code.

USER INTERFACE:

The design I had in mind was very simple and intuitive, also provides room for easy modifications

This is my first actual design of UI, only the individual stat checker (Judgement) has real UI:



Implemented:



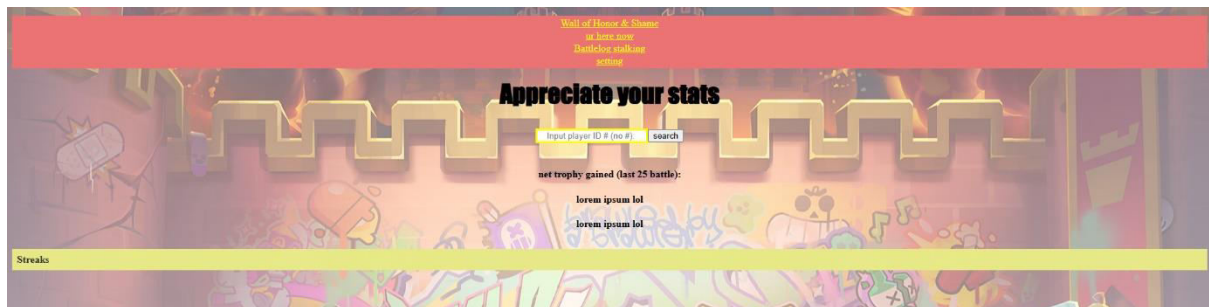
Later data will show up where the 25 is



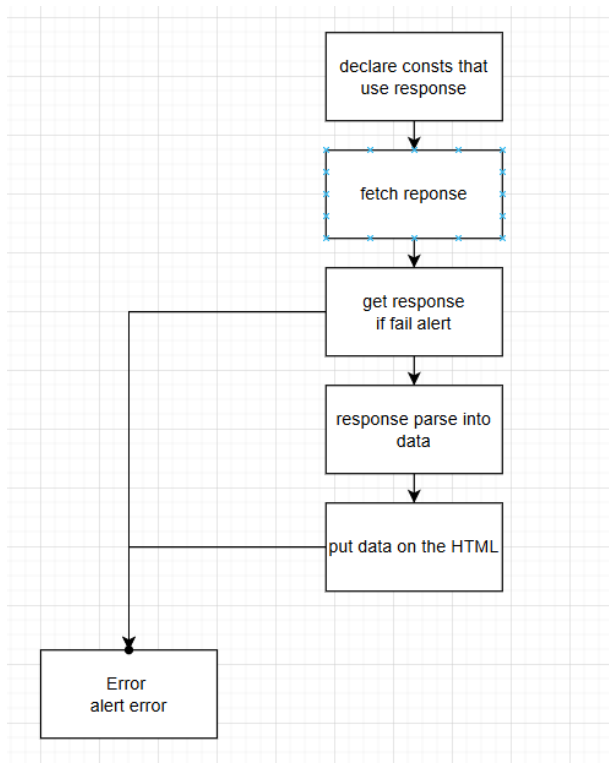
Each of the items on my navBar links to one of the Servlet access UIs

Later variations:

Made everything more aesthetically pleasing



Nonetheless, the JS behind the UI is also very straightforward, each of them sends a specific request to the servlet, at times triggering a response:



Database :

Using my IDEA environment, I can connect to the embedded database even when the server is not running, allowing me to see my tables. There were no arguing what to put in the database, it was decided from the very start

Bad brawler

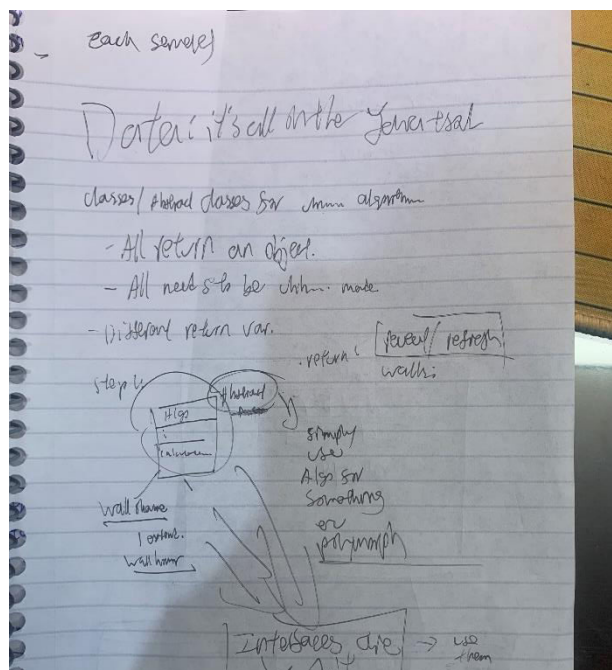
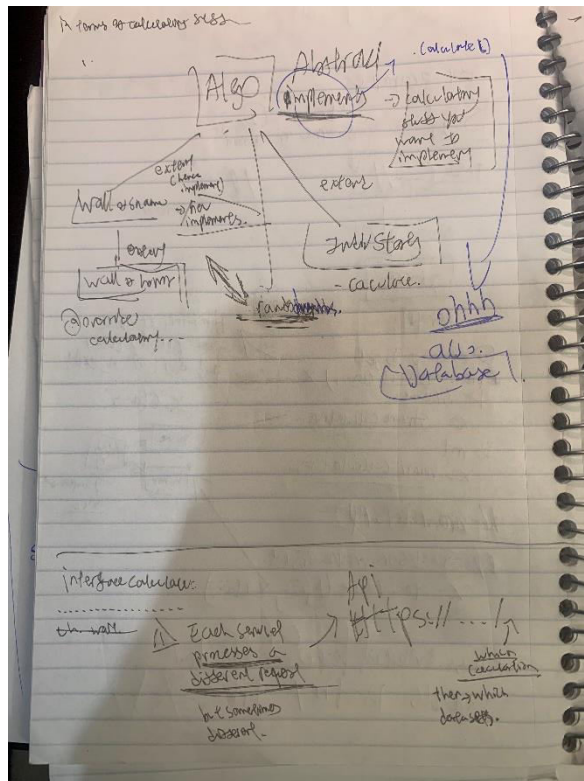
	Brawler
1	KIT
2	LILY
3	EDGAR

ID

	nickname	ID
1	大皇蜂+	PRYQQLRJV
2	yeahboijohncena	GLoQGRGR
3	reallyrealistic	YY99RLQP
4	paintbrush	20UQ0VC8V
5	DragonMaster21	80QQ9YRGL

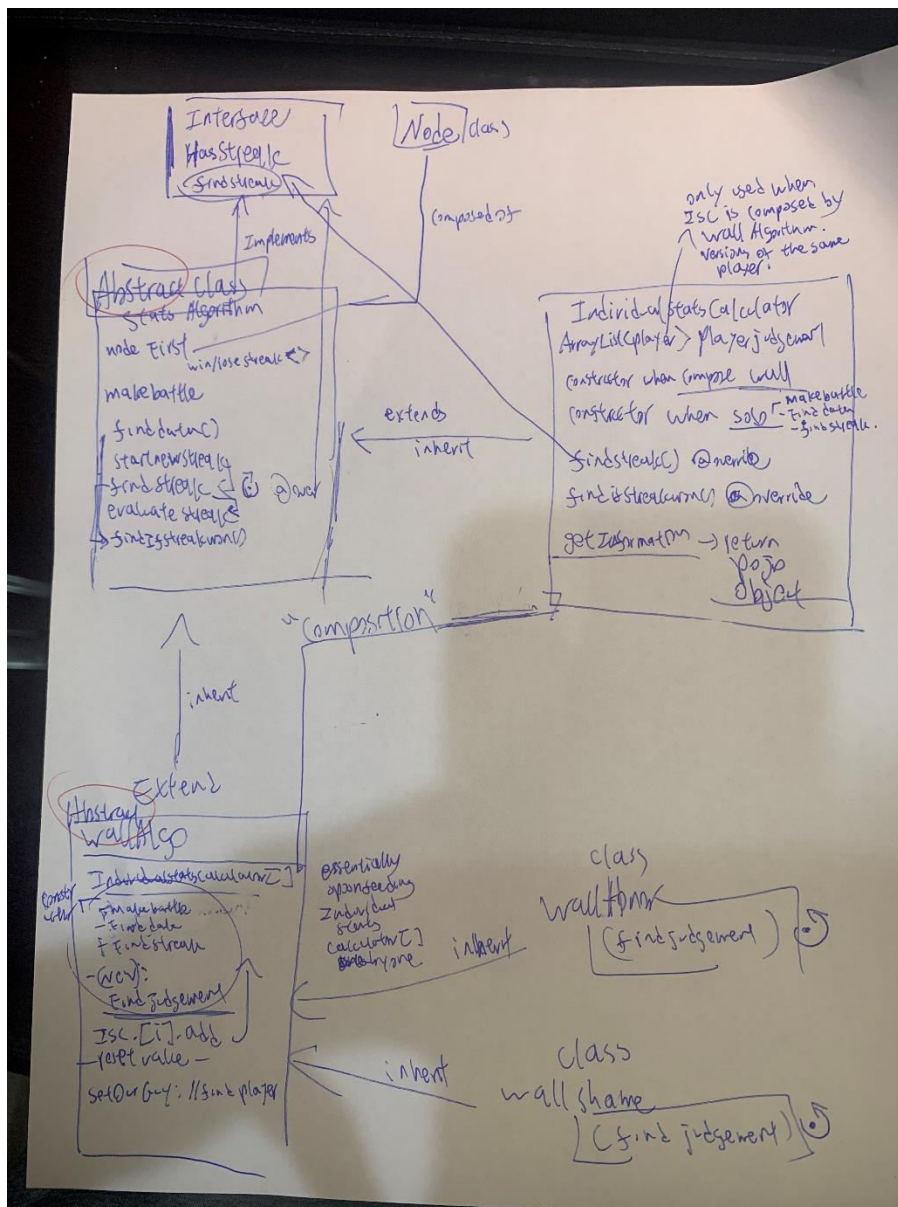
Algorithm:

When servlets access to calculate an Algorithm, most of the processes are similar, therefore my class interdependence was very much thought upon.



There is a interface called HasStreak, it has final variables that determines how many wins/losses counts as a streak. Anyone who opens the server files will be able to change that, affecting the operation of the whole program as expected. It makes extensions of the program trivial.

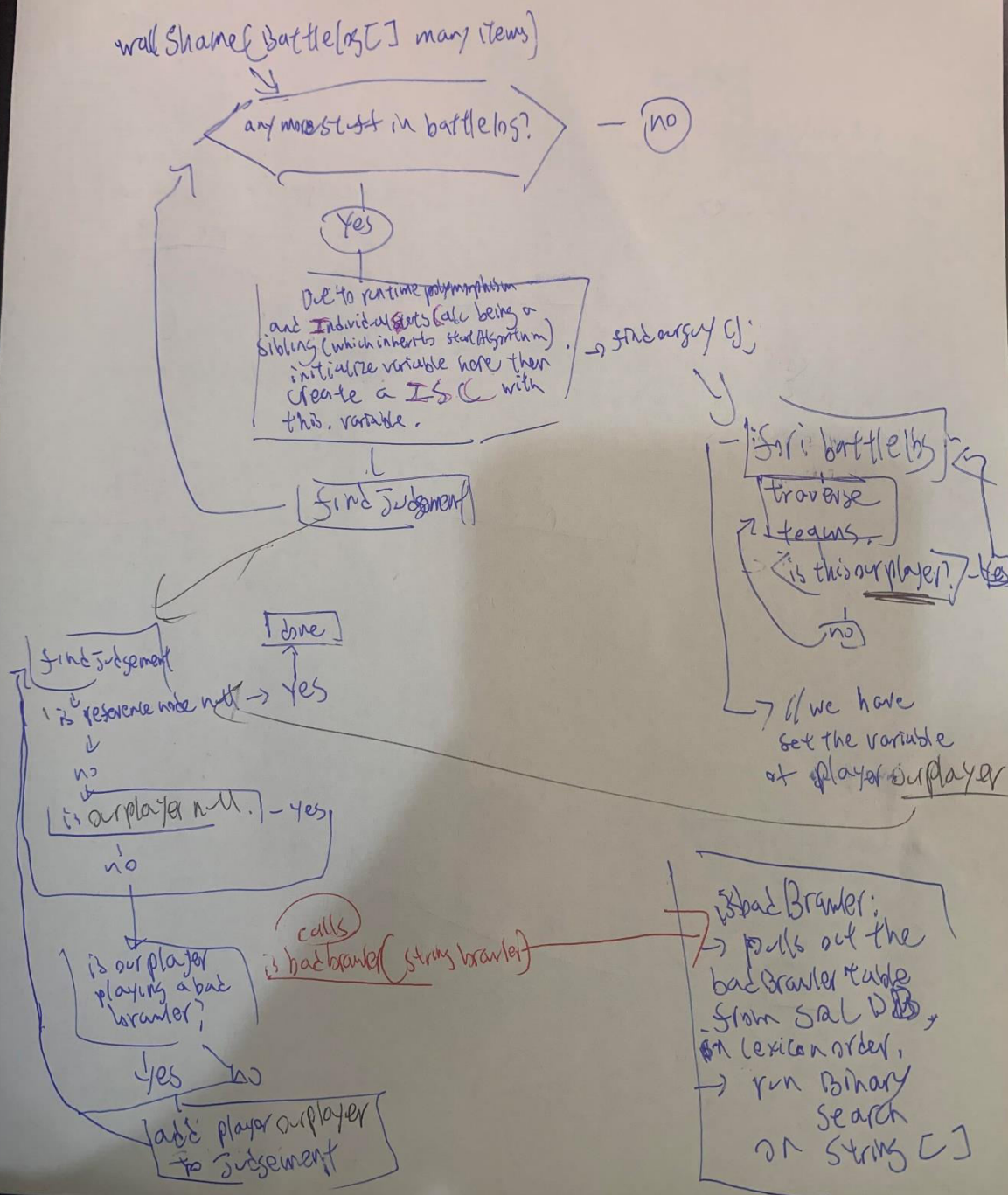
The UML:



The methods in the UML are highly modular and there are a lot of inheritance and overriding of methods.

I will briefly explain the flowchart of the most complex operation, which is findJudgement of WallShame class, which inherits ~~abstract~~ (now public due to needs) class WallAlgorithm, which inherits abstract class StatsAlgorithm.

process of findJudgement

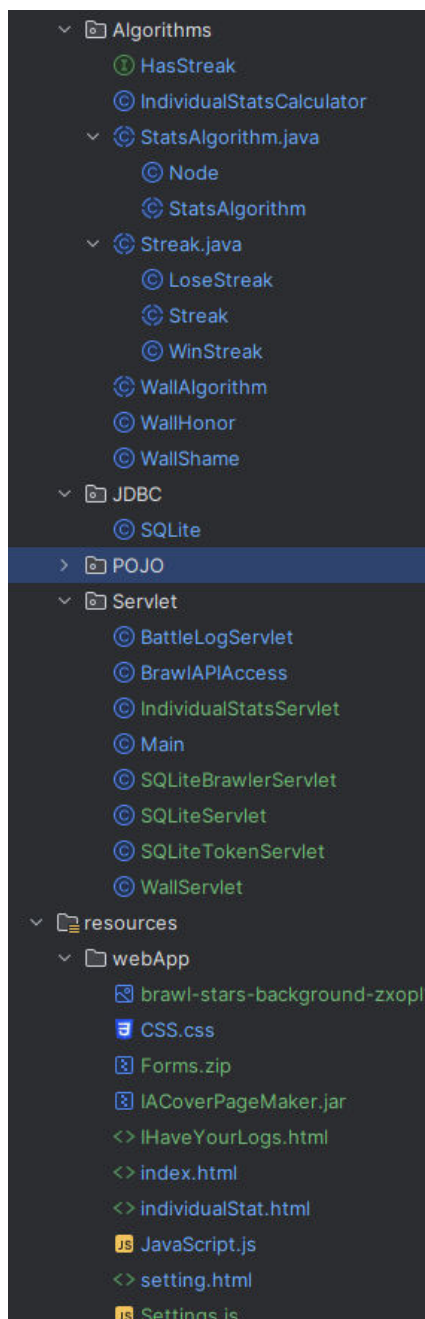


When iterating method findJudgement() in its constructor @overwrote by WallShame, it is a very complex and thought out process involving lots of modular methods as seen above. Both public wall classes uses the super constructor to create themselves (and said constructor also use super methods (from StatsAlgorithm) to gather data, but it doesn't affect findJudgement). When the constructor iterates through findJudgement, finally uses the WallShame algorithm isBadBrawler to

help out its request. Any of one of these methods could be reedited to suit my client's need easily, and the methods are very easy to understand.

Modularity:

My final structure is highly modular, with different packages for different parts of the program like thus:



The class methods also have different access modifiers, with those with protected or lower not intended to be used outside of package. This also ensures future implementations does not end with tears.

Test Plan

(Every new version) push buttons on HTML.index to see JS and backend response (UI criteria)	If HTML responds with information obtained via the server, the test is successful.
Maven test Algo: Ensure Jackson ObjectMapper can always retrieve POJO from JSON of Brawl Star API (API criteria)	If console could get non-null elements of POJO, the test is considered successful.
Maven test Algo: Insert console logs of sout(variable) in StatsAlgorithm and run sample outputs. Count the # of times console log was invoked (time complexity criteria)	If the proportion is exponentially proportional to # of input, it means the time complexity is messed up and the test is considered failure. If the proportion is logarithmically proportional to # of input or at least quadratically (eval teams), the test is considered success.
Start randomly inserting on the UI when test 1 is successful (Criteria on error and <i>trauma</i> tolerance)	If backend did not respond in a destructive manner to the DBMS and no unintended methods were invoked, test is considered successful. (JavaScript ignored bad request)
Maven test DatabaseFile Invoke statement and resultSet methods to insert and retrieve data (criterion on embedded database)	If changes were visible in database UI upon reset or if results were accurately inputted into console, test is considered successful. *NEW If database throws SQLite is locked error, please check if any statement was not closed. Test would be considered a failure immediately
Maven test on Algo Run subclasses of StatsAlgorihtm and serialize them into 2nd type POJO. And print them as om.ValueToString (ensure criterion 7 and 8 methods are functional)	If JSON file includes elements that are visibly of Brawl Star nature, test is considered successful.
Access program in School with School WIFI	If product is able to connect to Brawl Stars API by outputting expected results. We have

(criterion 3)	managed to sneak brawl stars in school, test is considered a massive success.
Run separate project with HTML and Javascript that parses JSON response into tables, open sample website of html.index .	If table contains elements from JSON, the JS method that gets table elements from JSON is functional, test would be successful.